

ПРАКТИЧНЕ ЗАНЯТТЯ №11

Тема: Оброблення виняткових ситуацій. Різновиди патернів Factory

Звіт про виконання практичного заняття

Виконав: Вівчар Вадим Вікторович

Група: АЛК-43

Мета роботи

Ознайомитися з принципами оброблення виняткових ситуацій у C++, розглянути призначення та різновиди патернів Factory, зокрема Factory Method та Abstract Factory, а також навчитися пояснювати їх застосування на конкретних прикладах.

Теоретичні відомості

Оброблення виняткових ситуацій використовується для реагування програми на помилки або нестандартні ситуації, які виникають під час виконання. До таких ситуацій можна віднести некоректні вхідні дані, помилки відкриття файлів, неправильний вибір типу об'єкта, відсутність потрібного ресурсу або неможливість виконати певну операцію.

У мовах об'єктно-орієнтованого програмування виняткові ситуації зазвичай обробляються за допомогою конструкцій try, catch і throw. Блок try містить код, у якому може виникнути помилка. За допомогою throw генерується виняток, а блок catch перехоплює його та виконує відповідну обробку.

Патерни Factory застосовуються для організації створення об'єктів. Вони дозволяють приховати від клієнтського коду конкретні класи, складну логіку створення об'єктів і перевірку параметрів. Це особливо зручно у великих програмах, де об'єкти можуть створюватися залежно від умов, типу вхідних даних або вибору користувача.

До основних різновидів патернів Factory належать Фабричний метод і Абстрактна фабрика. Фабричний метод зазвичай відповідає за створення одного типу об'єкта через спеціальний метод. Абстрактна фабрика використовується тоді, коли потрібно створити цілу групу пов'язаних між собою об'єктів без прив'язки до їх конкретних класів.

Хід роботи

Для виконання практичного заняття було підготовлено чотири програмні приклади мовою C++. Перший приклад демонструє базове оброблення виняткової ситуації під час ділення чисел. Другий приклад показує реалізацію патерну Factory Method для створення різних типів документів. Третій приклад демонструє патерн Abstract Factory для

створення сімей пов'язаних об'єктів інтерфейсу. Четвертий приклад поєднує фабрику класів із обробленням винятків у разі неправильного типу об'єкта.

У першому прикладі функція ділення перевіряє значення дільника. Якщо дільник дорівнює нулю, програма може згенерувати виняток `invalid_argument`. На скриншоті показано коректне виконання програми без виникнення винятку.

У другому прикладі клієнтський код не створює об'єкти документів напряду, а отримує їх через фабричний метод. Це дозволяє змінювати конкретний тип документа без зміни основної логіки програми.

У третьому прикладі абстрактна фабрика створює набір пов'язаних об'єктів інтерфейсу для різних платформ. Для Windows створюються `WindowsButton` і `WindowsCheckbox`, а для Mac - `MacButton` і `MacCheckbox`.

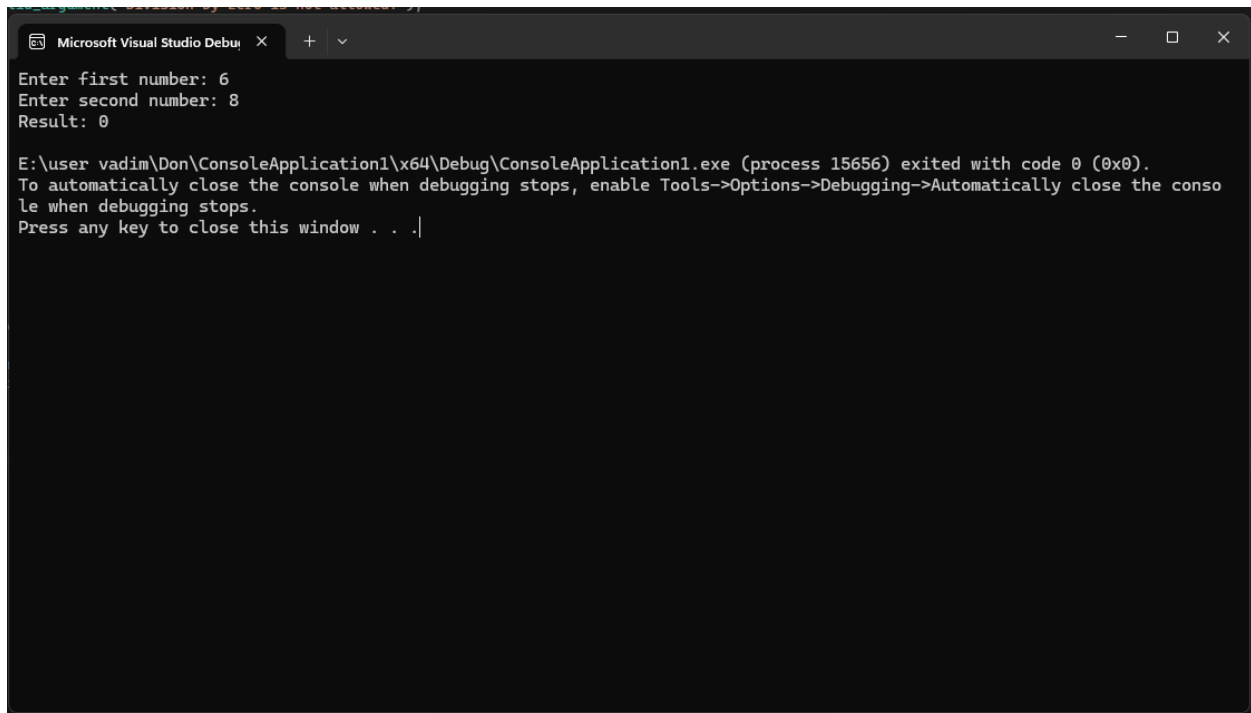
У четвертому прикладі фабрика створює об'єкт способу оплати за введеним типом. Якщо користувач вводить непідтримуваний тип, фабрика генерує виняткову ситуацію та виводить повідомлення про помилку.

Робоча таблиця з відповідями на контрольні запитання

№	Контрольне запитання	Відповідь	Конкретний приклад
1	Що виконує фабричний метод у класі?	Фабричний метод у класі відповідає за створення об'єктів. Він дозволяє не створювати об'єкти напряму через конструктор, а винести процес створення в окремий метод. Завдяки цьому клієнтський код працює зі спільним інтерфейсом або базовим класом і не залежить від конкретної реалізації.	У програмі для обробки документів метод <code>createDocument()</code> може створювати об'єкти <code>PdfDocument</code> або <code>WordDocument</code> . Клієнт викликає лише фабричний метод і отримує готовий документ.
2	Навіщо може бути потрібен фабричний метод у деякому класі? Які проблеми можна вирішити за його допомогою?	Фабричний метод потрібен тоді, коли програма повинна створювати об'єкти різних класів, але клієнтський код не повинен знати, який саме клас буде використано. Він допомагає приховати складну логіку створення, зменшити дублювання коду, спростити додавання нових типів об'єктів і централізовано обробляти помилки створення.	У системі оплати фабрика може створювати <code>CardPayment</code> , <code>CashPayment</code> або <code>PayPalPayment</code> . Якщо передано невідомий тип оплати, фабрика генерує виняток <code>invalid_argument</code> .
3	Яка структура патерну Фабричний метод?	Структура патерну складається з таких елементів: <code>Product</code> - спільний інтерфейс або базовий клас; <code>ConcreteProduct</code> - конкретні класи продуктів; <code>Creator</code> - клас, який оголошує фабричний метод; <code>ConcreteCreator</code> - клас, який реалізує або перевизначає фабричний метод і створює конкретний продукт.	Наприклад, <code>Payment</code> - спільний інтерфейс, <code>CardPayment</code> і <code>CashPayment</code> - конкретні продукти, <code>PaymentCreator</code> - базовий творець, а <code>OnlinePaymentCreator</code> - конкретний творець.
4	У чому зручність для клієнта замість безпосереднього створення потрібних об'єктів створювати для цього фабрику?	Клієнтському коду не потрібно знати назву конкретного класу, параметри конструктора та внутрішню логіку створення. Клієнт звертається до фабрики й отримує готовий об'єкт через спільний інтерфейс. Це робить програму простішою для підтримки та розширення.	У програмі доставки клієнт може викликати <code>createDelivery("courier")</code> , а фабрика сама створить об'єкт <code>CourierDelivery</code> . При додаванні нового способу доставки змінюється фабрика, а не весь клієнтський код.

№	Контрольне запитання	Відповідь	Конкретний приклад
5	Що може приховувати фабрика класів від клієнта?	Фабрика класів може приховувати назви конкретних класів, логіку вибору об'єкта, перевірку параметрів, залежності між класами, початкове налаштування об'єктів і оброблення виняткових ситуацій. Клієнт отримує результат і працює з ним через інтерфейс.	Фабрика повідомлень може створювати EmailMessage, SmsMessage або PushMessage. Якщо тип повідомлення неправильний, фабрика може згенерувати UnsupportedOperationException.
6	Чому патерн Фабричний метод інколи звуть віртуальним конструктором?	Фабричний метод називають віртуальним конструктором, тому що він виконує роль конструктора, але може бути перевизначений у підкласах. На відміну від звичайного конструктора, фабричний метод може повертати об'єкти різних конкретних класів залежно від ситуації.	Базовий клас ReportCreator може мати метод createReport(). Клас PdfReportCreator створює PDF-звіт, а ExcelReportCreator - Excel-звіт. Об'єкт створюється через метод, який можна перевизначити.
7	Які відмінності патерну "Абстрактна фабрика" від патерну "Фабричний метод"?	Фабричний метод зазвичай створює один тип об'єкта через спеціальний метод. Абстрактна фабрика створює цілу сім'ю пов'язаних об'єктів без зазначення їх конкретних класів. Тому Фабричний метод зручний для створення одного продукту, а Абстрактна фабрика - для набору взаємопов'язаних продуктів.	Фабричний метод може створювати один об'єкт Button. Абстрактна фабрика може створювати набір елементів інтерфейсу: WindowsButton, WindowsCheckbox, WindowsMenu або MacButton, MacCheckbox, MacMenu.

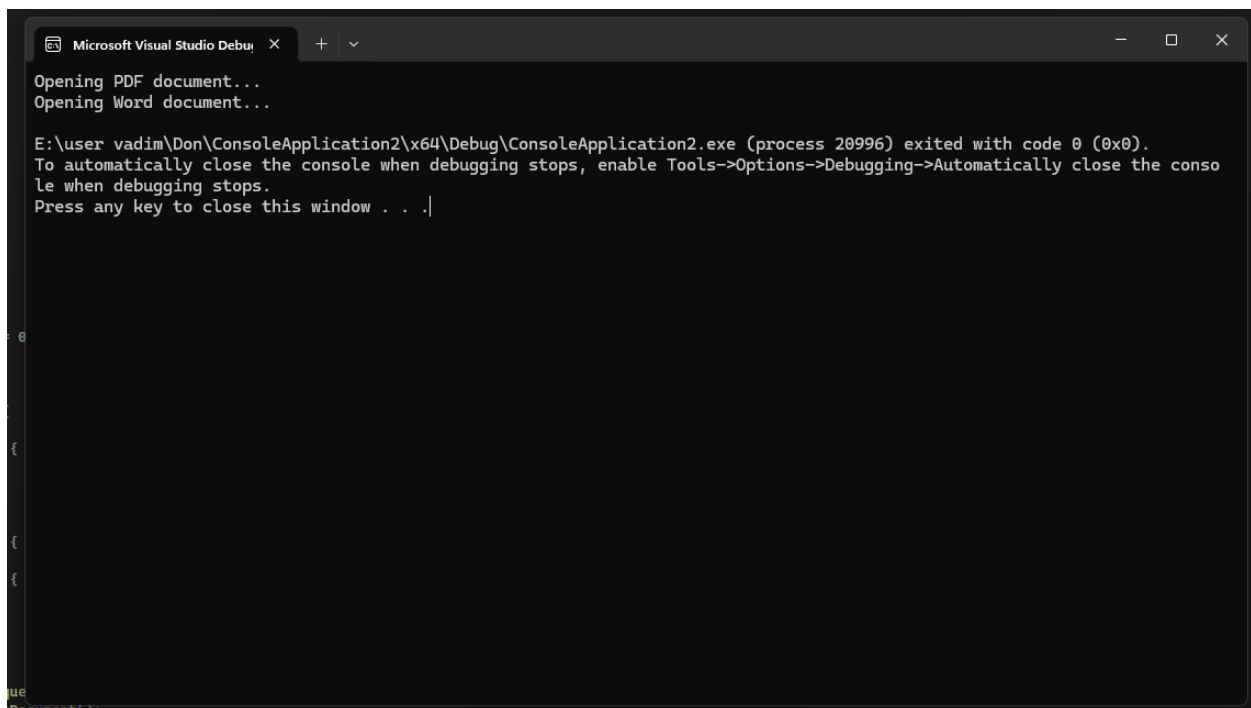
Результати виконання програм



```
Microsoft Visual Studio Debug Console
Enter first number: 6
Enter second number: 8
Result: 0

E:\user vladim\Don\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 15656) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 1 - Результат виконання програми з обробленням виняткових ситуацій



```
Microsoft Visual Studio Debug Console
Opening PDF document...
Opening Word document...

E:\user vladim\Don\ConsoleApplication2\x64\Debug\ConsoleApplication2.exe (process 20996) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 2 - Результат виконання програми з патерном Factory Method

```
Microsoft Visual Studio Debug Console
Windows interface:
Rendering Windows button...
Rendering Windows checkbox...

Mac interface:
Rendering Mac button...
Rendering Mac checkbox...

E:\user vadim\Don\ConsoleApplication3\x64\Debug\ConsoleApplication3.exe (process 6216) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 3 - Результат виконання програми з патерном *Abstract Factory*

```
Microsoft Visual Studio Debug Console
Enter payment type (card, cash, paypal): 3
Enter amount: 43
Error: Unsupported payment type: 3

E:\user vadim\Don\ConsoleApplication4\x64\Debug\ConsoleApplication4.exe (process 2028) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 4 - Результат виконання фабрики з обробленням виняткової ситуації

Висновок

У ході виконання практичного заняття було розглянуто призначення оброблення виняткових ситуацій та різновиди патернів Factory. Практичні приклади показали, що винятки дозволяють коректно реагувати на помилки під час виконання програми, а фабричні патерни допомагають відокремити логіку створення об'єктів від клієнтського коду.

Фабричний метод доцільно використовувати тоді, коли потрібно створювати об'єкти різних класів через спільний інтерфейс. Абстрактна фабрика застосовується у випадках, коли необхідно створювати цілі групи пов'язаних об'єктів. Також фабрики зручні під час оброблення виняткових ситуацій, оскільки вони дозволяють централізовано перевіряти вхідні дані та повідомляти про помилки створення об'єктів.